

Project Deep Dive: BigTree Discord RAG Bot

This document covers overall design, architecture, and module responsibilities. Full feature inventory: `FEATURE_LIST.md`; quick start: `SETUP_AND_TEST.md`.

Table of contents

- 1. Design rationale
 - What problem does this solve?
 - Why RAG instead of fine-tuning?
 - Four core design principles
 - Answer boundaries (deliberate non-goals)
 - Core Q&A flow
 - How knowledge becomes “like you”
 - 2. Technical architecture
 - Layered overview
 - Startup sequence (`bot/main.py`)
 - Main Cogs loaded at startup
 - Q&A main path (detailed)
 - How RAG and confidence work together
 - External dependencies and local state
 - Background tasks
 - Admin and client surfaces (optional)
 - 3. Feature landscape
 - 4. Core module notes
 - Config and entrypoint
 - Ingestion layer
 - Message handling and RAG
 - Review and learning loop
 - Content and news automation
 - Acquisition and promotion
 - Safety and operations
 - Admin and clients
 - 5. Key technical concepts
 - 6. Cost structure
 - 7. Safety and reliability design
 - 8. Project file structure
 - 9. Everyday command cheat sheet
 - 10. Slash Commands cheat sheet
 - 11. Environment install and initialization
-

1. Design rationale

What problem does this solve?

You run a large Discord stock/signal community where members ask questions around the clock. The channel owner cannot be online 24/7, but still wants:

1. Automatic replies that sound like their own tone and views (not generic AI finance boilerplate)
2. Uncertain answers to go through human review first — avoid wrong answers and invented entry points

3. A knowledge base that keeps growing (owner posts, approved reviews, YouTube transcripts, etc.)
4. Ops automation: daily/weekly summaries, Jin10 flash news, video digests, promo scheduling, new-member conversion

While also handling rate limits, spam/scam filtering, off-topic control, and keeping OpenAI cost and latency under control.

Why RAG instead of fine-tuning?

Approach	Pros	Issues for this project
Fine-tuning	Tone can be baked into weights	Views change daily; every new batch needs retraining; expensive and slow to iterate
RAG (this project)	New posts usable as soon as ingested; answers are traceable to history; easy to shut off / canary	Depends on retrieval quality and prompt constraints

Conclusion: store your historical content in ChromaDB + have GPT answer under a style guide. Style comes from data/style_profile.txt (generated by ingestion.analyze_style), not from changing model weights. Early planning also explicitly deferred “fine-tune first”.

Four core design principles

1. Retrieval-augmented, not invented

Embed and retrieve relevant history before generating. If retrieval is empty or distances are too far, hand off to the owner rather than forcing an answer.

2. Triple confidence gate, conservative by default

LLM self-score CONFIDENCE: 1-10 (default threshold 7) + retrieval quality (no context / best_distance too large) + hard domain rules (e.g. “can I enter now / give me a level” signal questions always go to review — the Bot cannot see the live tape).

3. Human-in-the-loop keeps getting sharper

- Owner posts / voice in target channels → auto-ingest
- Review Approve / Edit → Q&A ingested
- Reject → negative samples injected into later system prompts
- Member [up][down], Edit → feedback and knowledge-gap learning queue

4. Capabilities are switchable and canary-able

Most subsystems default off (Jin10, YouTube, daily/weekly summaries, AutoMod, Admin, etc.) and turn on via env; new capabilities also use Feature Flags + per-channel Canary to avoid whole-server blowups.

Answer boundaries (deliberate non-goals)

- Do not pretend to see the tape: signal / entry-timing questions always go to Owner review.
- Do not interrupt chitchat: default RESPOND_MODE=questions — only questions, @Bot, replies to Bot, images/voice, etc.
- Do not inject promo copy randomly: purchase intent takes a separate CTA path, split from RAG.
- Price levels are redacted: dangerous price wording is redacted after generation; optional Safety Guardrails (force review or add disclaimer).

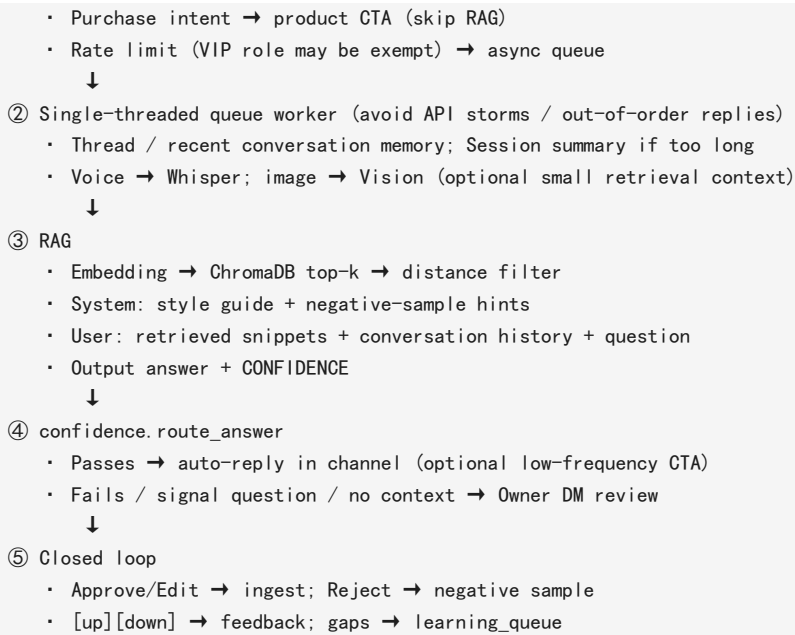
Core Q&A flow

Member asks: “What do you think of AAPL?”

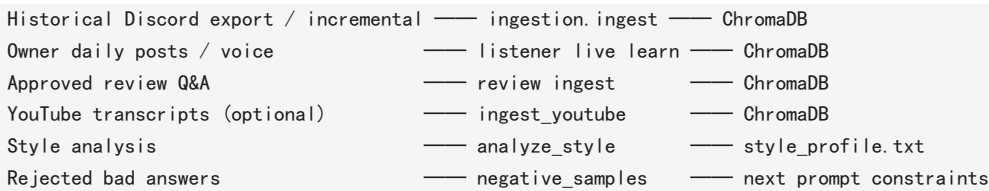


① on_message pre-checks

- Target channel? Excluded channel? Bot? Courtesy/spam/external links?
- Owner post → learn/ingest only; skip Q&A

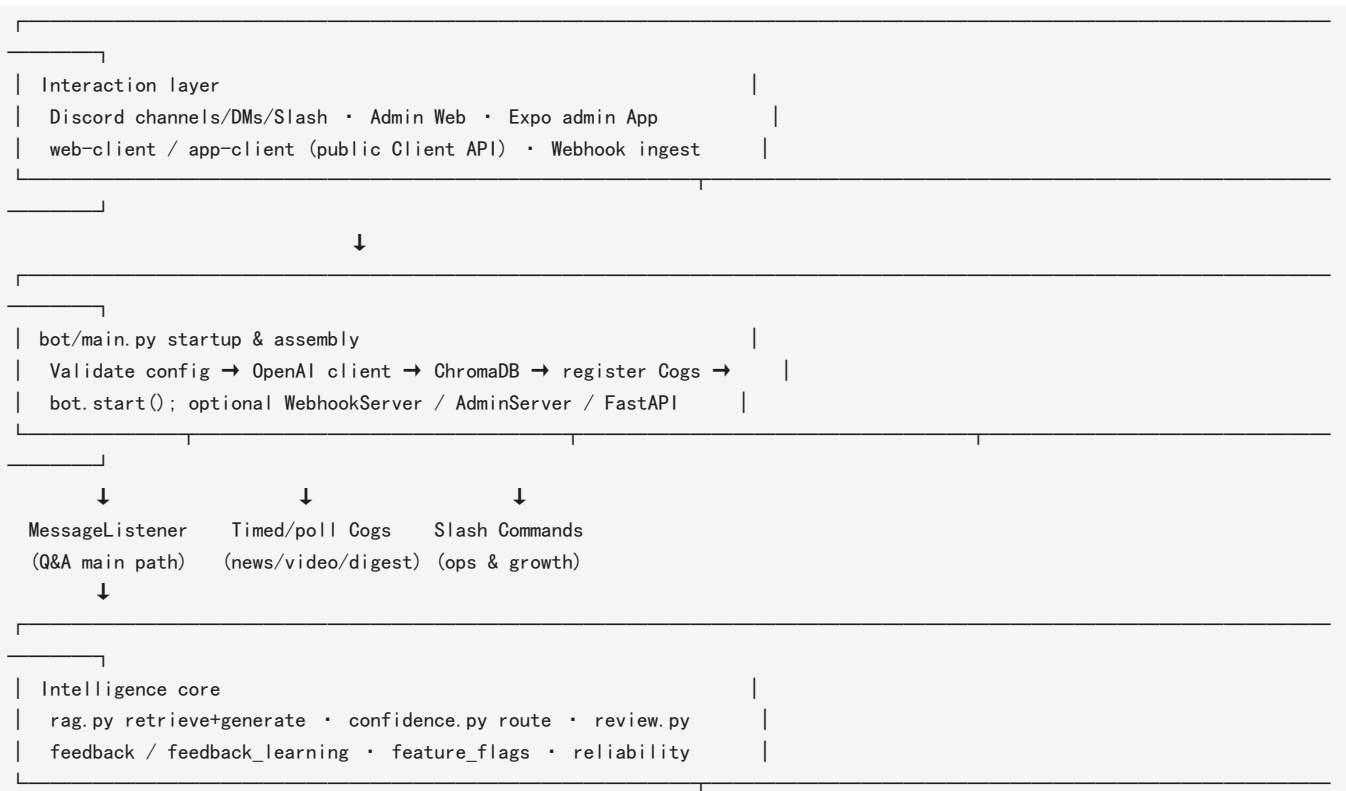


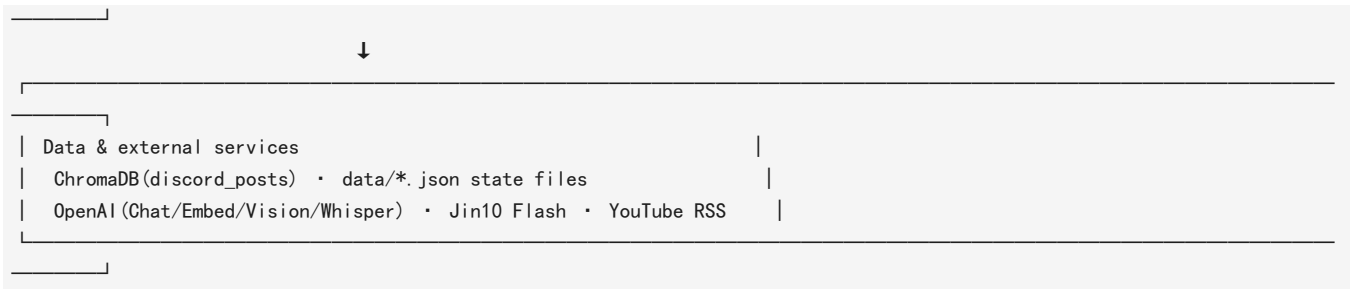
How knowledge becomes “like you”



2. Technical architecture

Layered overview





Startup sequence (bot/main.py)

1. `_validate_config()`: at least `DISCORD_BOT_TOKEN`, `OPENAI_API_KEY`
2. Create `openai.AsyncOpenAI` (timeout/retry wrapped again at the RAG layer)
3. Inject OpenAI into ban-words / Topic Guard; load `ban_words`
4. `chromadb.PersistentClient(CHROMADB_PATH)` → collection `discord_posts` (cosine) → `AsyncCollection` wrapper
5. Create Discord Bot (needs `message_content` / `members` / `invites` intents)
6. Register `on_ready` (guild-level + global Slash sync)
7. Load Cogs in order (see table below)
8. Start Webhook / Admin HTTP / FastAPI per switches
9. `bot.start(...)`; process signals trigger graceful unload

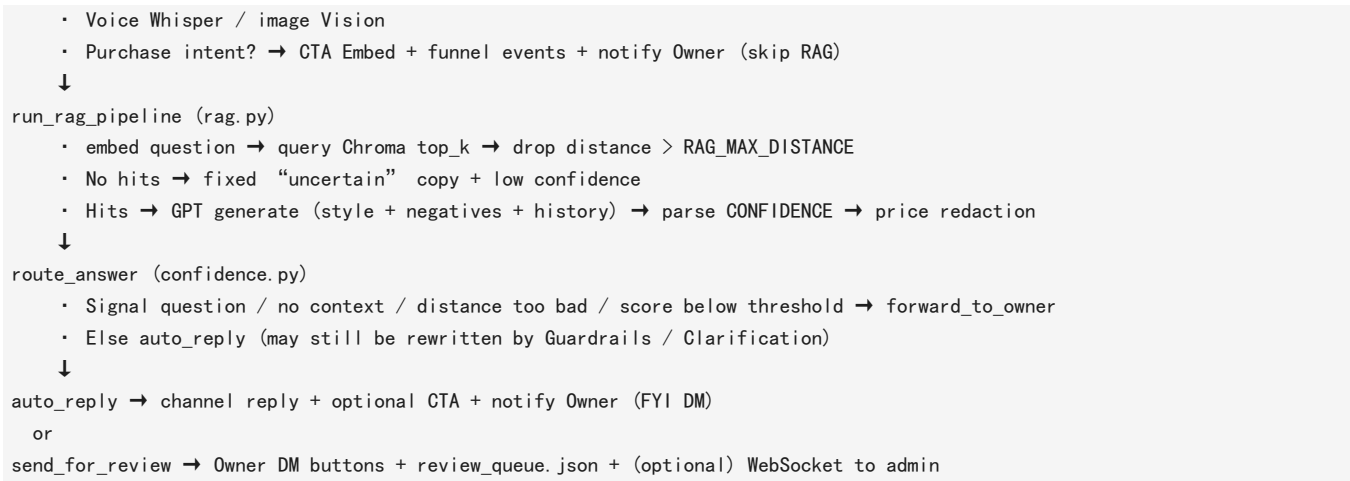
After `MessageListener` is ready it also: starts the message-queue worker, optional offline backfill, and periodic flush of `last_seen`.

Main Cogs loaded at startup

Order	Cog	Role
1	<code>MessageListener</code>	Channel listen, RAG Q&A, offline backfill, learn Owner posts
2 - 3	<code>PromotionCommands</code> / <code>BotCommands</code>	Slash: promotion + ops
4	<code>SchedulerCog</code>	Promo / lessons / reminders / FAQ push / SLA tick
5	<code>HealthCog</code>	Health and status
6	<code>IngestionSchedulerCog</code>	Periodically run <code>ingestion.ingest</code> + <code>analyze_style</code>
7	<code>DigestCog</code>	Channel activity digest (optional)
8	<code>NewsFeedCog</code>	Jin10 flash poll + offline backfill
9	<code>YouTubeMonitorCog</code>	New videos → schedule / ingest / summary
10	<code>AcquisitionCog</code>	Welcome drip, invite tracking
11	<code>PromoMonitorCog</code>	Source-channel activity → auto-schedule promos
12 - 13	<code>WeeklySummaryCog</code> / <code>DailySummaryCog</code>	Weekly/daily highlight summaries
14	<code>AutoModCog</code>	Spam/scam, ban words, topic limits

Q&A main path (detailed)

```
Discord on_message
|
|— Owner text/voice (target channel) → learn/ingest → return
|— Update last_seen; keyword alerts / testimonial collection (side paths)
|— _should_skip (bots, excluded channels, courtesy, spam, non-question mode, etc.)
|— Rate limit (non-VIP) → asyncio.Queue
↓
Single worker _handle_message
· Assemble Thread / channel conversation memory (optional Session summary)
```



How RAG and confidence work together

Stage	Key config / behavior
Retrieval	EMBEDDING_MODEL (default text-embedding-3-small), RAG_TOP_K (default 8), RAG_MAX_DISTANCE (default 0.6)
Generation	LLM_MODEL (default gpt-4o-mini); Vision uses VISION_MODEL (default gpt-4o)
Style	style_profile.txt, cache TTL ~5 minutes
Self-score	Trailing CONFIDENCE: N; parse failure treated as score 3
Routing	Default CONFIDENCE_THRESHOLD=7; best_distance 0.95 still reviews even if score is high; context_count==0 reviews
Clarification	FEATURE_CLARIFICATION_FOLLOWUP: low confidence that would still auto-reply can become a clarifying follow-up

External dependencies and local state

Type	Content
Discord	Gateway, messages, DMs, Slash, roles/invites, attachments
OpenAI	Chat, Embedding, Vision, Whisper; ban-word / topic semantic helpers
Jin10	Flash API polling (NEWS_POLL_INTERVAL_SECONDS; important flashes first)
YouTube	Channel RSS; yt-dlp/captions or Whisper ingest; GPT summary
ChromaDB	CHROMADB_PATH, collection discord_posts
data/*.json	Cursors and ops state: last_seen, jin10_last_id, review_queue, feedback, negative_samples, promos, funnel, stats, etc.

Background tasks

Module	Trigger	What it does
NewsFeedCog	Poll + backfill on every on_ready	Jin10 → Discord; advance last_id only after successful post
YouTubeMonitorCog	Timed / daily check	New video → lesson schedule + optional ingest + summary channels
DailySummaryCog / WeeklySummaryCog	ET clock	Scan Owner posts → GPT → push (dedupe / catch-up)
SchedulerCog	~every 60s	Due promos/lessons/reminders, FAQ push, SLA

PromoMonitorCog	Owner messages in source channel	Cancel old auto-promos; schedule new repeating pushes
AcquisitionCog	Member events + drip	Welcome sequence, invite attribution
IngestionSchedulerCog	Interval job	Subprocess incremental ingest + style re-analysis
Listener backfill	Startup when enabled	Answer questions missed while offline; learn missed Owner posts

Admin and client surfaces (optional)

Surface	Entry	Purpose
Admin Web	ADMIN_ENABLED + ADMIN_PORT	Browser status / config / KB
FastAPI admin API	API_ENABLED (often on by default)	For app/ Expo admin: review, KB, config; WebSocket pending reviews
Public Client API	CLIENT_API_ENABLED	/api/public/*: chat, news, summaries, search, etc.
web-client / app-client	Vite / Expo	End-user UI (no Discord login)

The core Q&A path does not depend on these surfaces: with API/Admin off, a pure Discord Bot still runs standalone.

3. Feature landscape

Grouped by capability (numbers match entries in FEATURE_LIST.md for cross-reference):

A. Intelligent Q&A core

Capability	Highlights
RAG Q&A	ChromaDB + GPT, style-guide driven
Confidence routing	Auto-reply / Owner review
Offline backfill	After restart, answer missed questions in target channels
Auto-learn Owner messages	Channel-owner posts ingested
Thread / image / voice	Thread context; Vision; Whisper transcription
Multilingual detection	Reply in Chinese / English / Japanese / Korean, etc.
Clarification / Session summary	Low-confidence clarifying follow-ups; long-chat memory compression
Feature Flags + Canary	Per-channel gray rollout of new capabilities

B. Content and news automation

Capability	Highlights
Jin10 flash news	Poll Flash API; important Embeds; reconnect backfill; advance last_id only after successful post
Daily / weekly summaries	Scan owner posts; GPT digest; dedupe and miss catch-up
YouTube Monitor	RSS new video → lesson schedule + optional ingest + summary channels; /resend_summary
Digest	Daily activity digest (separate channel)

C. Acquisition and promotion

Capability	Highlights
Promo / trial / lesson schedules	Repeat: once / hourly / daily / weekly / monthly

Promo Monitor	Source-channel events → auto daily push (default ET 16:00)
Welcome drip	Immediate welcome → value content → product CTA → day-3 reminder
Purchase-intent conversion	Price/subscribe keywords → product Embed + CTA + notify Owner
Invite virality /invite	Personal invite link + reward threshold
Opt-in notify-role DMs	Only consented roles can be DMed; /promo_notify_panel
/funnel	Conversion funnel board

D. Safety and quality

Capability	Highlights
Auto Mod	Keywords / ban words (exact + semantic) / links / invites / spam
Topic Guard	Selected channels: investment-only discussion (GPT classify)
Safety Guardrails	High-risk wording → force review or disclaimer
SLA monitoring	p95 latency, OpenAI error rate, review backlog alerts
Negative-feedback learning	Reject / downvote → negative samples and KB gap pool

E. Management surface

Admin Web, FastAPI API, Expo Admin App, Web/mobile clients, Webhook ingest, KB snapshots, conversation export, satisfaction and leaderboards, etc. Full list in `FEATURE_LIST.md`.

4. Core module notes

Config and endpoint

`bot/config.py` — reads all switches and thresholds from `.env` (RAG, channels, news, YouTube, summaries, acquisition, AutoMod, Feature Flags, SLA, etc.).

`bot/main.py` — validate secrets → init OpenAI / ChromaDB → register Cogs → guild-level slash sync (commands visible soon after restart) → Discord login.

Common core config examples:

Setting	Description
<code>TARGET_CHANNEL_IDS</code>	Channels RAG listens on
<code>CONFIDENCE_THRESHOLD</code>	Confidence threshold (default 7)
<code>RESPOND_MODE</code>	auto / review / questions / mention_only
<code>NEWS_FEED_ENABLED</code> / <code>NEWS_CHANNEL_IDS</code>	Jin10 push
<code>DAILY_SUMMARY_*</code> / <code>WEEKLY_SUMMARY_*</code>	Daily/weekly summary timing and channels
<code>YOUTUBE_MONITOR_ENABLED</code>	New-video monitoring
<code>AUTO_MOD_ENABLED</code>	Auto moderation
<code>WELCOME_FLOW_ENABLED</code>	New-member welcome and drip

Ingestion layer

`ingestion/preprocess.py` — preprocessing

Discord export JSON → Q&A pair extraction → merge consecutive messages → text cleanup → token chunking.

`ingestion/ingest.py` — vectorization

Batch Embedding → write ChromaDB; supports incremental (no duplicates).

ingestion/analyze_style.py — style analysis

Stats on length, common phrases, emoji, etc. → data/style_profile.txt for RAG prompts.

ingestion/ingest_youtube.py — YouTube ingest

Captions preferred; otherwise yt-dlp + ffmpeg + Whisper. Binaries may live under .venv/Scripts/; code resolves full paths and does not rely on system PATH. PDF ingest also supported (ingest_pdf.py).

Message handling and RAG

bot/listener.py — message orchestrator

1. Filter: bots, excluded channels, empty content, etc.
2. AutoMod / Topic Guard: spam and off-topic first
3. Intent: purchase intent → acquisition CTA (may skip RAG)
4. Rate limit: per-user cooldown + global cap; VIP roles exempt
5. Multimodal: voice notes Whisper; images Vision
6. Queue: asyncio.Queue serial processing to avoid API storms
7. Owner post learning, offline backfill, Thread context, reaction feedback collection

bot/rag.py — retrieve and generate

Retrieve: embedding → Top-K → distance filter → dedupe.

Generate: style guide + context + (optional) negatives / Session summary → LLM → parse CONFIDENCE.

Includes embedding cache, retries, and reliability instrumentation.

bot/confidence.py — routing

Typical triple condition (all configurable): has context, best distance close enough, confidence $\geq$ threshold. All must pass for auto-send; otherwise review. Can stack with Clarification and Guardrails.

Review and learning loop

bot/review.py + bot/review_queue.py

Low-confidence drafts DM Owner: Approve / Edit / Reject. Approve may ingest; Reject / Edit enter negatives or learning queue. Queue has expiry cleanup to avoid false SLA “backlog” alerts.

bot/feedback.py / bot/feedback_learning.py

[up][down] record satisfaction; periodically summarize Top N knowledge gaps to drive KB backfill.

Content and news automation

bot/news_feed.py — Jin10 flash news

- Default ~30s poll; NEWS_IMPORTANT_ONLY can push only important flashes
- Dedupe, ad filter; important messages use Embeds
- Every on_ready (including reconnect) backfills the offline window (default 24h, max 50 important)
- Advance last_id only after successful post — avoids misses

bot/daily_summary.py / bot/weekly_summary.py

- Scan configured channels for owner messages and replies; GPT Embed; optional @everyone
- Daily: prevent same-day double send (early timer wake will not double-post)

- Weekly: read failures retry and catch-up so “no messages” is not a false miss

bot/youtube_monitor.py

- Timed channel RSS check; new videos can schedule lesson posts, ingest, summarize to YOUTUBE_SUMMARY_CHANNELS
- Transcription failure can still send a new-video notice; /resend_summary can backfill summaries (auto KB / Whisper ingest / title fallback)
- Summaries may attach acquisition CTA buttons

bot/views_summary.py + /views

Server-wide scan of recent owner posts; short views summary posted to the current channel (no @everyone).

Acquisition and promotion

Module	Role
promo_config.py	CTA / product Embed / welcome-copy helpers
scheduler.py	Send promos, lessons, reminders when due
promo_monitor.py	Source-channel events → daily schedules
testimonials.py	Praise detection and review forwarding
welcome_flow.py	Welcome DM + multi-day drip (persisted in welcome_drip.json)
acquisition.py / acquisition_cog.py	Intent conversion, invite tracking, funnel
role_dm.py	Opt-in notify-role promo DMs (compliant bulk)

Detailed ops playbooks: GROWTH_PLAYBOOK.md, PROMOTION_GUIDE.md.

Safety and operations

Module	Role
auto_mod.py / ban_words.py	Multi-layer spam filter; ban words exact + semantic similarity
topic_guard.py	Topic limits on selected channels
guardrails.py	High-risk wording handling
reliability.py	SLA metrics and Owner alerts
health.py	Health-check Cog
feature_flags.py	Feature switches and canary channels
keyword_alert.py	Keyword DM alerts to Owner
kb_versioning.py	Knowledge-base snapshots

Admin and clients

Path	Description
bot/admin.py	Browser Admin panel
bot/api/	Admin API + public Client API + WebSocket
app/	Expo Admin mobile app
app-client/ / web-client/	End-user clients
bot/webhook.py	HTTP ingest

5. Key technical concepts

Embedding (vector embeddings)

Text → high-dimensional vectors; semantically similar text has nearby vectors. This project defaults to OpenAI embedding models.

ChromaDB

Local persistent vector store (chromadb_store/), similarity search over historical posts / video transcripts, etc.

Cosine Distance

Distance	Meaning
0.0	Nearly identical
~0.3	Highly related
~0.6	Marginally related
≥0.8	Usually filtered out

RAG

Retrieve trusted context first, then generate — reduces “pure model hallucination”.

Feature Flag / Canary

New capabilities can enable in a subset of channels first, then go full after stability is proven.

6. Cost structure

Item	Magnitude	Notes
Historical ingest (one-time / incremental)	Relatively low	Embedding billed by token
Each text Q&A	~\$0.001 class	Retrieval + GPT-4o-mini
Image Vision / voice Whisper	Higher	Per call / per duration
Jin10 polling	Near \$0	External HTTP; no OpenAI
Daily/weekly summaries, video digests	Medium	Depends on scanned volume and model
AutoMod semantic ban words / Topic Guard	Medium	Embeddings or GPT classification

Actual cost scales with question volume, multimodal share, and summary channel count. Rate limits and VIP exemption help control spend.

7. Safety and reliability design

Mechanism	Notes
Triple confidence checks	Lower chance of nonsense in public channels
Owner review queue	Uncertain content DMs first
Rate limit + VIP	Anti-abuse and cost control
AutoMod / Topic Guard	Spam, scams, off-topic
Guardrails	all-in / guaranteed returns → force review or disclaimer
Secrets only in .env	Not committed to Git
News last_id advances after success	Avoid offline misses
Daily dedupe / weekly retry catch-up	Avoid misses or double sends
SLA alerts	Latency, error rate, backlog notify Owner

8. Project file structure

```
treeProjectDiscordBot/
├── .env / .env.example
├── requirements.txt
├── Dockerfile / docker-compose.yml
├── keep_away.py
├──
├── bot/                                ← runtime
│   ├── main.py                        ← entrypoint
│   ├── config.py                      ← config
│   ├── listener.py                    ← message orchestrator
│   ├── rag.py / confidence.py / review*.py
│   ├── news_feed.py                  ← Jin10
│   ├── daily_summary.py / weekly_summary.py / views_summary.py
│   ├── youtube_monitor.py
│   ├── auto_mod.py / ban_words.py / topic_guard.py / guardrails.py
│   ├── acquisition*.py / welcome_flow.py / role_dm.py
│   ├── scheduler.py / promo_*.py / testimonials.py / commands.py
│   ├── reliability.py / feature_flags.py / feedback*.py
│   ├── api/ / admin.py / webhook.py
│   └── ...
├──
├── ingestion/                          ← offline ingest
│   ├── preprocess.py / ingest.py / analyze_style.py
│   └── ingest_youtube.py / ingest_pdf.py
├──
├── scripts/                            ← ops scripts (resend summaries, backtests, etc.)
├── tests/                              ← pytest
├── data/                               ← runtime JSON / style / state
├── docs/                               ← this guide and topic docs
├── app/ / app-client/ / web-client/
├── chromadb_store/
└── logs/
```

Topic docs: FEATURE_LIST.md, PROMOTION_GUIDE.md, GROWTH_PLAYBOOK.md, USER_GUIDE.md, CLIENT_USER_GUIDE.md, API_DESIGN.md, etc.

9. Everyday command cheat sheet

```
# Activate virtual environment
.venv\Scripts\Activate.ps1

# Start Bot
python -m bot.main

# Ingest Discord data (incremental)
python -m ingestion.ingest

# Ingest YouTube
python -m ingestion.ingest_youtube --urls "https://youtu.be/VIDEO_ID"

# Style analysis
python -m ingestion.analyze_style
```

```
# Tests
python -m pytest tests/ -v

# Knowledge-base document count (collection name may vary; often bigtree_knowledge)
python -c "import chromadb; c=chromadb.PersistentClient('./chromadb_store'); print([x.name for x in c.list_collections()])"

# Keep awake
python keep_aware.py
```

Resend YouTube summary (script fallback):

```
python scripts/resend_youtube_summary.py
python scripts/resend_youtube_summary.py --video-id VIDEO_ID
```

10. Slash Commands cheat sheet

Public

Command	Description
/ask	RAG question
/signal	Product intro
/invite	Personal invite link
/testimonials	Member testimonials
/faq	FAQ
/promo_notify	Claim/cancel event DM notifies

Owner — content and summaries

Command	Description
/daily_summary / /weekly_summary	Push daily/weekly summary now
/views	Short server-wide owner-views summary
/resend_summary	Resend YouTube GPT summary
/pin_summary	Summarize and pin recent discussion
/generate_faq	Generate FAQ

Owner — promotion and acquisition

Command	Description
/post_promo / /schedule_promo	Post/schedule promo (optional dm_role)
/schedule_trial / /schedule_lesson	Trial review / lesson
/list_promos / /cancel_promo, etc.	Manage schedules
/promo_notify_panel / /dm_role	Subscribe panel / opt-in role DM
/funnel	Conversion funnel

Owner — ops and quality

Command	Description
/status / /stats / /kb_report / /search_kb	Status and knowledge base
/satisfaction / /leaderboard / /ab_results	Feedback and experiments
/add_ban_word / /list_ban_words, etc.	Ban words
/add_alert / /list_alerts, etc.	Keyword alerts
/schedule_reminder, etc.	Custom reminders
/export_conversations / /kb_snapshots	Export and snapshots

Schedule repeat modes: once / hourly / daily / weekly / monthly.

Commands follow what the Bot currently registers; full descriptions in FEATURE_LIST.md.

11. Environment install and initialization

For a quick start see SETUP_AND_TEST.md.

Prerequisites

Component	Requirement	Verify
Python	3.11+	<code>python --version</code>
Node.js	18+ (frontends)	<code>node --version</code>
FFmpeg / deno	Recommended for YouTube Whisper path	May live under <code>.venv\Scripts\</code>
Discord Bot Token	Developer Portal	—
OpenAI API Key	OpenAI	—
Git	Version control	<code>git --version</code>

Install steps

```
# 1. System tools (example)
winget install --id Python.Python.3.11 -e
winget install --id Gyan.FFmpeg -e

# 2. Clone and enter project
git clone <your-repo-url>
cd treeProjectDiscordBot

# 3. Virtual environment
python -m venv .venv
.venv\Scripts\Activate.ps1

# 4. Dependencies
pip install -r requirements.txt

# 5. Config
Copy-Item .env.example .env
# Required: DISCORD_BOT_TOKEN, OPENAI_API_KEY, OWNER_USER_ID, TARGET_CHANNEL_IDS

# 6. Ingest + (optional) style
python -m ingestion.ingest
python -m ingestion.analyze_style

# 7. Start
python -m bot.main
```

Rebuild the virtual environment after moving machines (Windows)

Do not copy an old .venv. Rebuild with the py launcher:

```
Remove-Item -Recurse -Force .venv
py -m venv .venv
.venv\Scripts\Activate.ps1
python --version
python -m pip install --upgrade pip
pip install -r requirements.txt
python -m bot.main
```

If Activate.ps1 is blocked:

```
Set-ExecutionPolicy -Scope CurrentUser RemoteSigned
```

Verify install

```
python -m pytest tests/ -v  
python -c "import chromadb; c=chromadb.PersistentClient('./chromadb_store'); print([x.name for x in c.list_collections()])"
```

Healthy startup logs roughly include: OpenAI init, ChromaDB document count, Bot ready, message-queue worker start.

Frontends (optional)

```
cd app; npm install; npx expo start  
cd app-client; npm install; npx expo start  
cd web-client; npm install; npm run dev
```

See `USER_GUIDE.md`, `CLIENT_USER_GUIDE.md`.